Exam 3
CpE 3150-Intro to Microcontrollers and Embedded Systems

Question	Total Possible	Test Score
1	10	
2	6	
3	13	
4	7	
5	11	
6	12	
7	12	
8	17	
9	12	
Total	100	

This exam is open book, open notes.
Show all work. Partial credit will be given based on work shown.

This is an individual exam and not a group activity.

Sign and date below indicating that you completed this exam on your own and did not obtain or provide assistance to anyone else taking this exam.

Printed Name:_____ **Student #:**_____

Signature:_____

1. (10 Points) Given the following C program to generate/output a 1 kHz square wave on PB3 and at the same time transfer data from PORTD to PORTA. Convert the use of Timer 0 for generating the square wave which uses polling (below) to use the Timer 0 Clear Timer on Compare Match (CTC) interrupt mode. This will require you to: a) convert the Timer 0 mode of operation to CTC including the initialization of the Timer 0 registers TCNT0 and OCR0, b) configure Timer 0 CTC mode as an interrupt source, c) write the Interrupt Service Routine (ISR) for Timer 0 CTC mode, and d) modify the main while(1) loop to transfer data from PORTD to PORTA while generating the square wave on Timer 0, with Timer 0 CTC as an interrupt source. Do not forget to include needed header files.

```
// generate a 1 kHz square wave on PB3 and at the same transferring data from PORTD to PORTA

// TIMER0 FOR SQUARE WAVE
// PORTD => INPUT, PORTB => OUTPUT
//
// FOSC = 8 MHZ
// THALF = 0.5 X 1000 US = 500 US

// n = 500 us/[(1cc/1MC)(1s/8 10^6 cc)] = 4000 MCs

#include <avr/io.h>

int main(void)
{
	DDRB |= 0b00001000;  // PORTB output
	DDRD = 0x00;  // PORTD INPUT
	PORTD = 0xff; // PULLUP RESISTORS FOR PORTD
	DDRA = 0xff;  // PORTA output

	TCCR0 = 0x04; 0b00000100 for prescale of 256 for clk for ATmega32
	TCNT0 = -16;



	while(1)
	{

		// use TIFR for ATmega32
		while (!(TIFR & (1<<TOV0)))
		{
			PORTA = PIND;
		}

		TCCR0 = 0x00; // turn off timer 0 for reset

		TIFR = 0x01;   // write a 1 to TOV0 to reset overflow flag
		TCNT0 = -16;    // reset preload
		PORTB ^= 0x08; // 00001000
		TCCR0 = 0x04; // turn timer 0 back on
	}

	return 0;
}
```

SOLUTION

```
#include <avr/io.h>
#include <avr/interrupt.h>

ISR(TIMER0_COMP_VECT);

int main(void)
{
	DDRB |= 0b00001000;  // PORTB output
	DDRD = 0x00;  // PORTD INPUT
	PORTD = 0xff; // PULLUP RESISTORS FOR PORTD
	DDRA = 0xff;  // PORTA output


	TIMSK = 1<<OCIE0;
	TCNT0 = 0;
	OCR0 = 15;


	TCCR0 = 0b00001100; 0b00000100 for prescale of 256 for clk for ATmega32 with CTC mode

	SEI();
	while(1)
	{
		PORTA = PIND;
	}



	return 0;
}

ISR(TIMER_COMP_VECT) {

   PORTB ^= 0b00001000;
}
```

2. (8 Points) Given the following C program to generate/output a 1 kHz square wave on PB3 and at the same time transfer data from PORTD to PORTA. Convert the use of Timer 0 for generating the square wave which uses polling (below) to use the Timer 0 Overflow (TOV0) interrupt mode. This will require you to: a) convert the Timer 0 Overflow mode of operation without using interrupts to using/configuring Timer 0 Overflow as an interrupt source, b) write the Interrupt Service Routine (ISR) for Timer 0 Overvflow (TOV0) mode, and d) modify the main while(1) loop to transfer data from PORTD to PORTA while generating the square wave on Timer 0, with Timer 0 Overflow as an interrupt source. Do not forget to include needed header files.

```
// generate a 1 kHz square wave on PB3 and at the same transferring data from PORTD to PORTA

// TIMER0 FOR SQUARE WAVE
// PORTD => INPUT, PORTB => OUTPUT
//
// FOSC = 8 MHZ
// THALF = 0.5 X 1000 US = 500 US

// n = 500 us/[(1cc/1MC)(1s/8 10^6 cc)] = 4000 MCs

#include <avr/io.h>

int main(void)
{
	DDRB |= 0b00001000;  // PORTB output
	DDRD = 0x00;  // PORTD INPUT
	PORTD = 0xff; // PULLUP RESISTORS FOR PORTD
	DDRA = 0xff;  // PORTA output

	TCCR0 = 0x04; 0b00000100 for prescale of 256 for clk for ATmega32
	TCNT0 = -16;



	while(1)
	{

		// use TIFR for ATmega32
		while (!(TIFR & (1<<TOV0)))
		{
			PORTA = PIND;
		}

		TCCR0 = 0x00; // turn off timer 0 for reset

		TIFR = 0x01;   // write a 1 to TOV0 to reset overflow flag
		TCNT0 = -16;    // reset preload
		PORTB ^= 0x08; // 00001000
		TCCR0 = 0x04; // turn timer 0 back on
	}

	return 0;
}
```

SOLUTION

```
#include <avr/io.h>
#include <avr/interrupt.h>

ISR(TIMER0_OVR_VECT);

int main(void)
{
	DDRB |= 0b00001000;  // PORTB output
	DDRD = 0x00;  // PORTD INPUT
	PORTD = 0xff; // PULLUP RESISTORS FOR PORTD
	DDRA = 0xff;  // PORTA output


	TIMSK = 1<<TOIE0;
	TCNT0 = -16;


	TCCR0 = 0b00000100; 0b00000100 for prescale of 256 for clk for ATmega32 with CTC mode

	SEI();
	while(1)
	{
		PORTA = PIND;
	}



	return 0;
}

ISR(TIMER_OVR_VECT) {

   PORTB ^= 0b00001000;
   TCNT0 = -16;
}
```

3. (14 Points) Convert the assembly program (below) to C code (ATmega32) in the space on the left.

```
.ORG 0
RJMP MAIN

.ORG 0X0004
RJMP ISR_FN

.ORG 0x50
LDI R16, HIGH(RAMEND)
OUT SPH, R16
LDI R16, LOW(RAMEND)
OUT SPL R16
CBI DDRD, 3
SBI PORTD, 3
LDI R17, 0b00001100
OUT MCUCR, R17
OUT R17, 1<<INT1
OUT GICR, R17
LDI R17, 0XFF
OUT DDRC, R17
LDI R18, 0X00
OUT PORTC, R18
SEI
HERE: RJMP HERE


.ORG 0x100
ISR_FN: INC R18
OUT PORTC, R18
RETI
```

Put your C program code conversion from assembly program (shown on left) here.

```
#INCLUDE <avr/io.h>
#INCLUDE <avr/interrupt.h>

ISR(INT1_VECT);

INT MAIN() {

DDRD ^= 0b11110111;
PORTD |= 0b00001000;

MCUCR = 0b00001100;
GICR = 1<<INT1;
DDRC = 0xFF;
PORTC = 0X00;
SEI();
WHILE(1);

RET 0;
}

ISR(INT1_VECT) {
PORTC += 1;
}
```

4. Given the assembly language program below. Answer the questions below.

```
.ORG 0
RJMP MAIN

.ORG 0X0004
RJMP ISR_FN

.ORG 0x50
LDI R16, HIGH(RAMEND)
OUT SPH, R16
LDI R16, LOW(RAMEND)
OUT SPL R16
CBI DDRD, 3
SBI PORTD, 3
LDI R17, 0b00001100
OUT MCUCR, R17
OUT R17, 1<<INT1
OUT GICR, R17
LDI R17, 0XFF
OUT DDRC, R17
LDI R18, 0X00
SEI
HERE: RJMP HERE


.ORG 0x100
ISR_FN: INC R18
OUT PORTC, R18
RETI
```

a. **(3 Points**) What is the interrupt source used in this program? What is the port and port pin used with this interrupt source?

EXTERNAL INTERRUPT 1 PD3

b. (3 **Points**) What "event" causes this interrupt source to be triggered?

MCUCR => 11 => RISING EDGE OF PD3

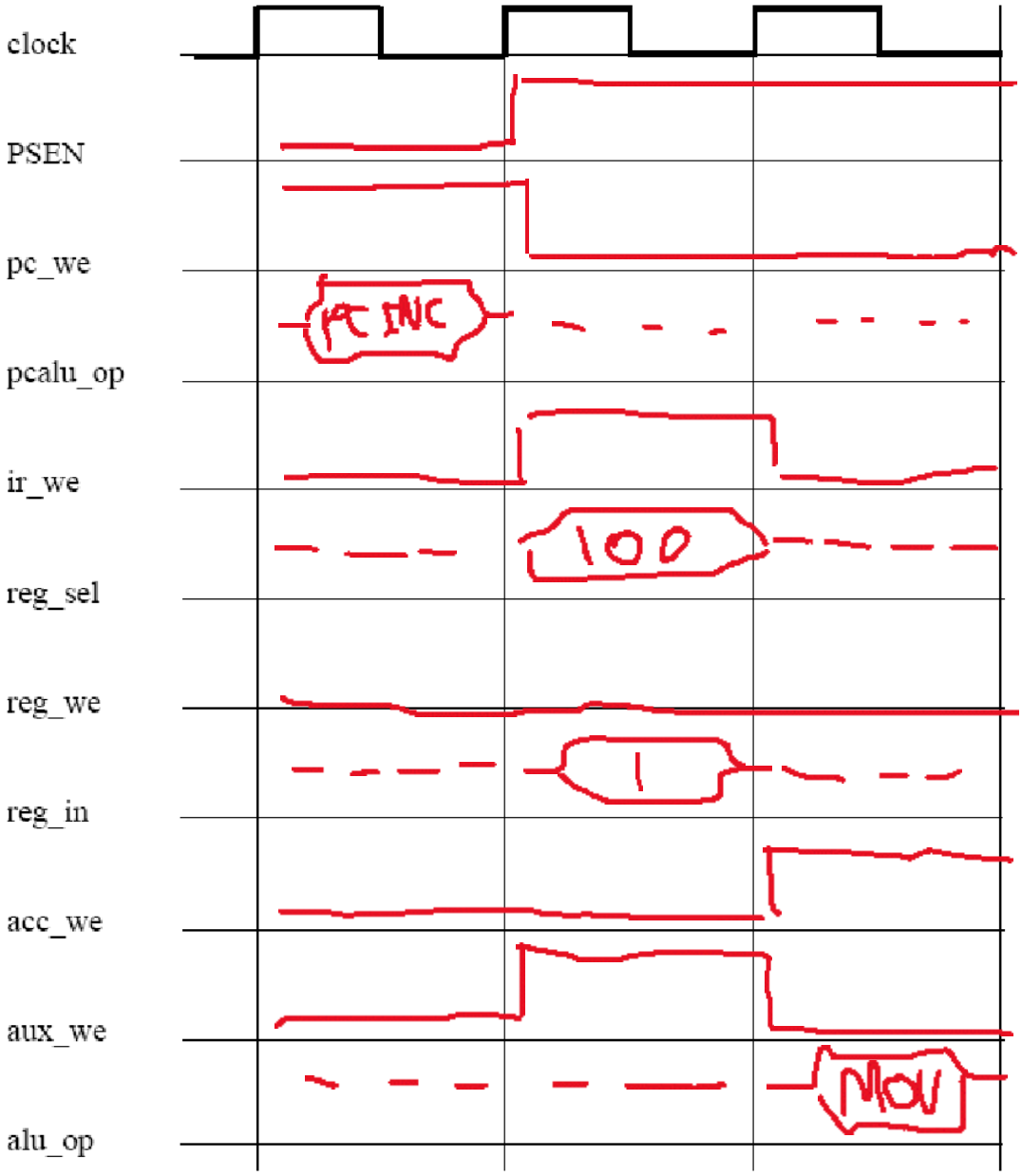
c. (**2 Points**) What action is performed in the interrupt service routine?

INCREMENTS COUNT OF RISING EDGE EVENTS IN PC

5. **(10 Points)** Draw the timing diagram for the following control signals on the diagram below when the **WIMP51** is executing the instruction "**MOV A, R4**". Label the fetch, decode and execute parts of the instruction cycle on the clock portion of the signal below.

The dotted lines have been drawn as a graphical reference only and do not reflect the value of the corresponding signal. If you think a particular control signal is of no significance during a particular time period, indicate this opinion by using a **dashed** line to denote the signal as a "don't care" for that time period. Give "command" values for pcalu_op and alu_op; for example, indicate the value of "pcalu_op" as "PC_INC" or "PC_ADD" rather than representing this command by a numeric code. If you are not sure about a particular control signal, make an educated guess, indicate that the value you have marked is a guess, and explain why you made this particular guess. Remember, PSEN/ pulses low whenever a byte is being fetched from code memory.

The WIMP51 architecture schematic and instruction set are available on the following pages.



Wimp51 Instructions

MOV A, #D	01110100	dddddddd	A<=D
ADDC A, #D	00110100	dddddddd	C,A<=A+D+C
MOV Rn, A	11111nnn		Rn<=A
MOV A, Rn	11101nnn		A<=Rn
ADDC A, Rn	00111nnn		C,A<=A+Rn+C
ORL A, Rn	01001nnn		A<=A or Rn
ANL A, Rn	01011nnn		A<=A and Rn
XRL A, Rn	01101nnn		A<=A xor Rn
SWAP A	11000100		$A<=A_{(3-0)}\&A_{(7-4)}$
CLR C	11000011		C<=0
SETB C	11010011		C<=1
SJMP rel	10000000	aaaaaaaa	PC<=PC+rel+2
JZ rel	01100000	aaaaaaaa	PC<=PC+rel+2 if Z

WIMP51 ARCHITECTURE SCHEMATIC

